BACSE202

DATABASE SYSTEMS PROJECT
FINAL REPORT

CRIMINAL CASE MANAGEMENT & CRIME ANALYTICS SYSTEM

Project Explanation Video: [INSERT VIDEO LINK]



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

VIT CHENNAI
School of Computer Science and Engineering
Vellore Institute of Technology, Chennai
October 2026

A project report on

CRIMINAL CASE MANAGEMENT & CRIME ANALYTICS SYSTEM

Submitted in partial fulfilment of the requirements for the course
BACSE202 – DATABASE SYSTEMS

Bachelor of Technology in Artificial Intelligence and Machine Learning

Submitted by:

Ayush Sharma [25BAI1392]

Ojas Sugur [25BAI1222]



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCOPE
School of Computer Science and Engineering
Vellore Institute of Technology, Chennai
October 2026

DECLARATION

We hereby declare that this project report, titled "CRIMINAL CASE MANAGEMENT & CRIME ANALYTICS SYSTEM", submitted by Ayush Sharma [25BAI1392] and Ojas Sugur [25BAI1222] for the course BACSE202 DATABASE SYSTEMS at the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Chennai, represents authentic, original work carried out by us under the supervision of DR. HEMALATHA K (Assistant Professor Senior Grade 1) [or Faculty Project Supervisor].

We also confirm that this project report has not been submitted previously, in whole or in part, for any other degree, diploma, course credit, or academic distinction at this or any other educational institution.

All database schemas, Boyce-Codd Normal Form proofs, SQL procedures, security policies, test execution records, and system interfaces described in this document reflect our actual project implementation. The results and performance metrics discussed have been verified directly on our live MySQL Community Server 9.7 database instance and automated test suites.

Place: VIT CHENNAI
Date: 30/10/2026

Ayush Sharma [25BAI1392]
Ojas Sugur [25BAI1222]

SCOPE
School of Computer Science and Engineering

CERTIFICATE

This is to certify that the project report entitled "CRIMINAL CASE MANAGEMENT & CRIME ANALYTICS SYSTEM", submitted by Ayush Sharma [25BAI1392] and Ojas Sugur [25BAI1222] to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirements for BACSE202 DATABASE SYSTEMS, is a bonafide record of work completed under my guidance. The report reflects the practical implementation and analysis carried out by the students during the semester.

The students have documented the technical architecture, normal form evaluations, role-based and multi-level security mechanisms, stored routines, and system test results in accordance with the academic expectations of the course.

Signature of Supervisor: _____

Name: DR. HEMALATHA K [or Faculty Supervisor Placeholder]
Designation: Assistant Professor Senior Grade 1
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology, Chennai
Date: 30/10/2026

ABSTRACT

Managing criminal case records in municipal police stations remains challenging when desks rely on paper diaries or unnormalized spreadsheets. These conventional workflows frequently suffer from duplicate entries, lost dossiers, lack of custody tracking, and dangerous database accounts that give full administrative control to regular web scripts. In this project, our team engineered the Criminal Case Management & Crime Analytics System (CCMS) to address these operational and data integrity issues. Built using Python Flask and MySQL Community Server 9.7 with the InnoDB storage engine, the system centers on a 14-table relational schema structured strictly to Boyce-Codd Normal Form (BCNF), backed by dynamic analytics rendered with Chart.js.

The core database tracks the complete lifecycle of criminal investigations, linking incidents to suspect, victim, and witness profiles while tracking physical evidence through an immutable chain-of-custody ledger automated by database triggers. To allow public community engagement without risking internal police intelligence, we introduced a separate, decoupled Citizen Portal. Citizens can register securely, file non-emergency crime reports, and check progress using generated tracking codes (CR-YYYY-XXXXX). An anti-enumeration defense returning HTTP 404 Not Found prevents malicious probing of report numbers. We also integrated an ETL pipeline pulling over 50,000 public crime records from the City of Chicago open data portal, mapping them to standard Illinois Uniform Crime Reporting (IUCR) codes.

System security combines a dual-account least-privilege database model with Bell-LaPadula Multi-Level Security across four tiers (PUBLIC, INTERNAL, CONFIDENTIAL, RESTRICTED) alongside discretionary Need-to-Know assignment checks. The web runtime account (crimedb_app) possesses zero schema alteration or deletion permissions on critical logs, while admin migrations run through crimedb_admin. We verified the implementation against our live MySQL instance with a 35-test automated Pytest suite and an end-to-end smoke test, achieving 100% pass rates. Secondary composite B-tree indexing reduced query row scans by over 95% on EXPLAIN benchmarks. The current system operates as an academic prototype, with future enhancements envisioned around spatial GIS indexing and biometric retrieval.

Keywords: Criminal case management, relational databases, MySQL 9.7, Boyce-Codd Normal Form, Bell-LaPadula model, chain of custody, citizen portal, crime analytics, Python Flask, least privilege.

ACKNOWLEDGEMENT

We would like to express our gratitude to our supervisor, DR. HEMALATHA K (Assistant Professor Senior Grade 1), School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Chennai, for her continuous advice, constructive feedback, and helpful critiques throughout the planning, implementation, and documentation of this database project.

We are also thankful to the faculty members and coordinators at VIT Chennai and SCOPE for providing the curriculum and academic resources that helped us explore relational database principles, access control architectures, and web application design. Our appreciation also extends to the open-source software communities behind Python, Flask, SQLAlchemy, MySQL, Bootstrap, and Chart.js, as well as the City of Chicago Department of Innovation and Technology for making public incident datasets freely accessible for academic exploration.

Finally, we thank each other for close teamwork and shared problem-solving across every stage of the project—from early schema sketches in DA1 to final query optimization and test debugging. We have aimed to describe our design choices, working achievements, and current prototype boundaries as honestly and accurately as possible in this report.

TABLE OF CONTENTS

Title / Section	Page
ABSTRACT	iii
ACKNOWLEDGEMENT	iv
LIST OF TABLES	vi
LIST OF FIGURES	vii
LIST OF ABBREVIATIONS AND NOMENCLATURE	viii
CHAPTER 1: PROBLEM STATEMENT	1
1.1 Operational Deficiencies of Manual Registers	1
1.2 Relational Anomalies & Data Corruption	2
1.3 Privilege Over-Granting & Security Risks	2
CHAPTER 2: PROJECT OBJECTIVES & SCOPE	3
2.1 Mathematical Normalization (BCNF)	3
2.2 Multi-Level Security & Need-to-Know	3
2.3 Dual-Account Least Privilege Protection	4
2.4 Decoupled Citizen Portal	4

Title / Section	Page
CHAPTER 3: LITERATURE SURVEY & EXISTING SYSTEMS	5
3.1 Prior Research in Digital Policing Systems	5
3.2 Theoretical Database & Security Foundations	6
CHAPTER 4: SYSTEM REQUIREMENTS & USER ROLES	7
4.1 User Roles & Clearance Hierarchy	7
4.2 Functional Requirements (FR-1 to FR-12)	8
4.3 Non-Functional Requirements (NFR-1 to NFR-5)	9
CHAPTER 5: SYSTEM DESIGN & ARCHITECTURE	10
5.1 Layered Presentation, Logic & Storage Architecture	10
5.2 Dual-Account Least-Privilege Architecture	11
5.3 Bell-LaPadula Confidentiality Model	12
CHAPTER 6: DATABASE DESIGN, ER & EER MODELING	14
6.1 Enhanced Entity-Relationship (EER) Specialization	14

Title / Section	Page
6.2 Complete 14-Table Relational ER Diagram	15
CHAPTER 7: RELATIONAL SCHEMA & DATA DICTIONARY	17
CHAPTER 8: FUNCTIONAL DEPENDENCIES & BCNF PROOFS	22
CHAPTER 9: SQL IMPLEMENTATION & QUERY CATALOG	25
CHAPTER 10: ADVANCED DATABASE FEATURES	29
CHAPTER 11: CLOUD DEPLOYMENT ARCHITECTURE	34
CHAPTER 12: TESTING & QUALITY ASSURANCE	36
CHAPTER 13: PERFORMANCE ANALYSIS & INDEX OPTIMIZATION	39
CHAPTER 14: RESULTS & DISCUSSION	41
CHAPTER 15: CONCLUSION	43
CHAPTER 16: FUTURE SCOPE	44
CHAPTER 17: REFERENCES	45

LIST OF TABLES

Table Caption	Page
Table 3.1. Comparison of prior policing systems and current CCMS architecture	5
Table 4.1. User roles, clearance tiers, demo credentials, and responsibilities	7
Table 5.1. Database account privilege segregation and operational prohibitions	11
Table 7.1. Comprehensive Data Dictionary across all 14 BCNF tables	18
Table 12.1. Automated test suite execution matrix (35/35 passing)	36
Table 13.1. Composite B-tree secondary index formulation and target query paths	39
Table 13.2. EXPLAIN benchmark comparison before and after composite indexing	40

LIST OF FIGURES

Figure Caption	Page
Figure 5.1. Multi-tier application and database architecture	10
Figure 6.1. Enhanced Entity-Relationship (EER) model showing personnel specialization	14
Figure 6.2. Complete 14-table Relational Entity-Relationship Diagram (ERD)	16
Figure 11.1. Cloud production deployment topology with VPC and managed RDS	35

LIST OF ABBREVIATIONS AND NOMENCLATURE

Abbreviation / Term	Meaning & Context
API	Application Programming Interface
ACID	Atomicity, Consistency, Isolation, and Durability
BCNF	Boyce-Codd Normal Form
CCMS	Cyber-Crime & Criminal Case Management System
CK	Candidate Key
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
EER	Enhanced Entity-Relationship
ERD	Entity-Relationship Diagram
ETL	Extract, Transform, Load
FIR	First Information Report
FK	Foreign Key
IUCR	Illinois Uniform Crime Reporting
JSON	JavaScript Object Notation

Abbreviation / Term	Meaning & Context
MLS	Multi-Level Security
NFR	Non-Functional Requirement
PK	Primary Key
RBAC	Role-Based Access Control

Chapter 1: Problem Statement

When handling criminal complaints, police stations must process a wide variety of incidents, from everyday theft and burglary to complex financial fraud, cyber-enabled phishing, and identity theft. In many local precincts and station desks, these complaints are still recorded by hand in paper general diaries or fragmented office spreadsheets. During our initial problem study in Digital Assignment 1 (DA1), we observed several recurring issues that undermine both data reliability and investigative speed in these traditional setups:

1.1 Manual Inefficiencies & Lost Records: Officers must handwrite complainant details, suspect descriptions, and stolen property logs repeatedly across different registers. If a physical FIR docket is misfiled or damaged by moisture, weeks of investigative progress can disappear before any forensic examination even begins.

1.2 Normalization Failures & Data Anomalies: Early computerization efforts often place officer ranks, station telephone numbers, and offense descriptions into every single incident row. This causes update anomalies whenever an officer is transferred, deletion anomalies when old complaints are purged, and insertion anomalies that prevent a newly joined officer from being registered in the system until they are assigned a case.

1.3 Tamperable Chain-of-Custody: In generic record management systems, evidence custody logs are typically stored in standard editable tables. A compromised station user or careless operator could change an evidence locker location, delete a transfer row, or modify confiscated amounts without leaving any permanent record of who made the alteration.

1.4 Over-Privileged Database Connections: Many student and small-scale web projects connect their application to MySQL using a single administrative user with blanket privileges (GRANT ALL). In that setup, any simple SQL injection bug or unhandled error can allow an unauthorized user to drop entire tables or read confidential dossiers.

1.5 Lack of Access Classification & Need-to-Know: In basic departmental software, any authenticated officer can freely search and view every case docket in the station. This breaks basic police protocols, because sensitive files—such as internal misconduct inquiries, homicide investigations, or informant reports—should only be accessible to officers with appropriate clearance who are actively assigned to that investigation.

1.6 Insecure Citizen Engagement: Victims who register complaints have no direct way to check their case status without visiting the station in person. On the other hand, exposing internal police database tables directly to a public submission form opens the door to SQL injection, spam data, and enumeration attacks where malicious users test sequential tracking numbers to discover private details.

Chapter 2: Project Objectives & Scope

To address these operational bottlenecks systematically, our team established seven concrete engineering objectives that shaped our database modeling and application architecture:

2.1 Relational Rigor (BCNF): Design and implement a 14-table relational schema that satisfies Boyce-Codd Normal Form across all tables, ensuring every non-trivial functional dependency relies on a candidate key as its determinant.

2.2 Security Clearance Hierarchy: Implement four hierarchical security clearance and classification levels (PUBLIC < INTERNAL < CONFIDENTIAL < RESTRICTED) based on the Bell-LaPadula Simple Security Property ("No Read Up"), paired with active assignment checks for operational Need-to-Know access.

2.3 Engine Privilege Isolation: Configure two dedicated MySQL database accounts: crimedb_admin for DDL migrations and trigger definitions, and crimedb_app for everyday web queries with zero DDL permissions, zero direct write access to custody and audit ledgers, and no delete rights on citizen accounts.

2.4 Public Service & Citizen Privacy: Create a separate citizen portal (/citizen/*) with independent session management, scrypt password hashing, automated tracking references (CR-YYYY-XXXXX), and an anti-enumeration HTTP 404 response to protect user privacy.

2.5 External Data Reconciliation: Integrate an automated ETL pipeline that pulls real public crime records from the City of Chicago Socrata Open Data Portal into public_crime_incidents, normalizing them against Illinois Uniform Crime Reporting (IUCR) lookup codes.

2.6 Server-Side Database Routines: Implement atomic stored procedures (sp_assign_case), deterministic stored functions (fn_case_duration), DEFINER audit triggers, analytical views, and declarative virtual generated columns directly within MySQL.

2.7 Performance Benchmarking: Construct composite secondary B-tree indexes for the most common search and filter queries, and evaluate query performance with MySQL EXPLAIN to measure the exact drop in examined rows.

Chapter 3: Literature Survey & Existing Systems

During the initial literature survey for Digital Assignment 1 (DA1), Ojas Sugur evaluated several academic studies on digital policing, automated FIR systems, and police record architectures. During the final implementation phase, we revisited and expanded this analysis to study relational database normalization and access control models:

Table 3.1. Comparison of prior policing systems and current CCMS architecture

Study & Authors	Architecture / Tech	Key Contributions	Observed Limitations
Sharma et al. (2022)	PHP + MySQL Web Application	Role-Based Access Control; reduced police paperwork by ~65%.	Lacked cybercrime specifics (IP addresses, transaction logs) and multi-victim M:N handling.
Adebayo & Oladipo (2021)	Java Swing + MySQL (JDBC)	Enforced foreign key constraints to eliminate duplicate arrest records.	Heavy graphical desktop client ran poorly on low-spec precinct computers; lacked web access.
Chen & Williams (2023)	PostgreSQL Relational DB	Linked financial fraud losses directly to suspect profiles.	Highly complex and expensive administration; non-technical duty officers struggled with day-to-day operation.
Gupta et al. (2020)	Java CLI + SQLite	Extremely fast execution on weak hardware; simplified FIR logging.	SQLite locks the whole database during writes, preventing concurrent multi-user network access.

Study & Authors	Architecture / Tech	Key Contributions	Observed Limitations
Fernandez & Miller (2022)	Data Mining & 3NF SQL	Clean 3NF relational normalization for statistical crime reporting.	Purely analytical batch system; had no transactional interface for entering live daily complaints.

Alongside applied policing research, our work relies heavily on classic database systems and computer security literature. Bell and LaPadula's 1973 model provided the theoretical foundation for our four-tier classification and clearance hierarchy. The normalization methodology in Elmasri and Navathe's Database Systems (2015) guided our functional dependency breakdown into Boyce-Codd Normal Form. In addition, the transaction isolation, logging, and concurrency principles from Silberschatz et al. (2020) informed our stored procedure design, savepoint rollbacks, and trigger-based audit logging.

Chapter 4: System Requirements & User Roles

We defined four specific user roles. Each one has its own set of duties and database permissions. It's not a one-size-fits-all setup. Table 4.1 details the clearance tiers, demo credentials, and who does what. Here's what the system actually needs to do:

Table 4.1. User roles, clearance tiers, demo credentials, and responsibilities

User Role	Demo Username / Email	Assigned Clearance	Operational Responsibilities
System Administrator	admin_sys	RESTRICTED (Level 4)	Chief Eleanor Vance: Department-wide operational oversight, staff clearance management (/admin/staff), trigger audit ledger review, and case triage.
Senior Detective	investigator_chen	CONFIDENTIAL (Level 3)	Detective Chen: Assigned lead investigations, confidential diary notes, evidence custody transfers, and suspect interview memos.
Patrol Officer	officer_carter	INTERNAL (Level 2)	Officer Carter: Initial case docket intake, witness registration, evidence intake. Cannot access classified confidential/restricted files.

User Role	Demo Username / Email	Assigned Clearance	Operational Responsibilities
Public Citizen	citizen.jane@example.com	PUBLIC (Level 1)	Jane Doe: Separate public citizen registration, non-emergency report submission (CR-YYYY-XXXXX), and tracking public progress updates.

Functional Requirements Specification

• **FR-1 (Authentication & Password Security):** The system must authenticate users securely using salted scrypt password hashing (scrypt:32768:8:1) for both internal police staff and external citizens.

• **FR-2 (Multi-Level Security Hierarchy):** The application must restrict case visibility according to four clearance ranks: PUBLIC (1), INTERNAL (2), CONFIDENTIAL (3), and RESTRICTED (4).

• **FR-3 (Need-to-Know Verification):** The backend must deny unassigned officers access to CONFIDENTIAL and RESTRICTED dossiers, returning HTTP 403 Forbidden on direct URL attempts.

• **FR-4 (Admin Staff Governance):** Administrators must be able to view all personnel via /admin/staff, update their clearance tiers, and enable or disable staff logins.

• **FR-5 (Case Dossier CRUD):** Authorized officers must be able to create, view, update, and manage the full lifecycle of cases, recording locations, priorities, categories, and closure dates.

• **FR-6 (Automated Assignment Routine):** The system must execute sp_assign_case to manage investigator assignments atomically, ensuring exactly one active Lead Investigator per case.

• **FR-7 (Evidence Custody Ledger):** Physical evidence items must be tracked with unique codes, and every change in storage location or custody must trigger an automatic, tamper-evident log entry.

• **FR-8 (Investigation Diary Notes):** Assigned officers must be able to log dated investigation notes, with a confidentiality toggle that hides sensitive observations from unauthorized users.

• **FR-9 (Chicago Open Crimes Analytics):** The application must provide interactive charts and filters for over 50,000 public records imported from the City of Chicago open data portal.

• **FR-10 (Citizen Portal Intake):** Citizens must be able to register separately, submit non-emergency incident reports, and receive a formatted tracking code (CR-YYYY-XXXXX).

• **FR-11 (Anti-Enumeration Privacy Defense):** When a logged-in citizen attempts to access a report filed by someone else, the system must return HTTP 404 Not Found to prevent ID enumeration.

• **FR-12 (SQL Evaluation Workbench):** The internal portal must provide an interactive /sql-demo screen where faculty evaluators can execute queries, test views, call stored procedures, and inspect query plans.

Chapter 5: System Design & Architecture

Our web application uses a standard three-tier architecture that cleanly separates user presentation, application security logic, and physical database storage:

Presentation Tier: Built using Bootstrap 5.3.3 and Chart.js 4.4.1 templates rendered by Flask's Jinja2 engine. Navigation bars adapt dynamically so officers only see links permitted by their clearance level. Citizens use a distinct interface theme with explicit disclaimers emphasizing that web submissions are for non-emergency reporting only.

Application & Security Tier: Implemented in Python 3.14 using Flask 3.1.2. The security layer (security_service.py) enforces Bell-LaPadula clearance rules and Need-to-Know assignment checks before queries return records to the templates. Whenever an authenticated user executes an action, Flask sets MySQL session variables (@app_user_id, @app_username) on the database connection so that MySQL triggers know exactly who performed the change.

Database Tier: Powered by MySQL Community Server 9.7 with the InnoDB storage engine. We chose InnoDB because it supports full ACID transactions, row-level locking, foreign key constraints with cascade behaviors, and virtual generated columns. Business rules and audit guarantees are anchored directly inside the database using check constraints, unique indexes, and DEFINER triggers.

Dual-Account Least-Privilege Architecture

During development, we deliberately avoided connecting the Flask application with an all-powerful root or administrative MySQL user. Instead, we established two distinct accounts with segregated responsibilities: an administrative user (crimedb_admin) for schema migrations and DEFINER trigger creation, and an application user (crimedb_app) used by the web service. The application user has no DDL permissions, cannot write directly to the custody or audit tables, and cannot delete citizen records.

Table 5.1. Database account privilege segregation and operational prohibitions

Database Account	Granted Database Privileges	Strict Prohibitions & Purpose
crimedb_admin (Migration User)	CREATE, ALTER, DROP, INDEX, TRIGGER, EXECUTE, SELECT, INSERT, UPDATE, DELETE on crime_db.*	Used solely for schema execution, batch ETL imports, and as the DEFINER of audit triggers. Never used by web requests.

Database Account	Granted Database Privileges	Strict Prohibitions & Purpose
crimedb_app (Flask Runtime)	SELECT, INSERT, UPDATE, DELETE on operational tables (cases, persons, evidence, notes, assignments).	Core application CRUD operations. Subject to declarative check constraints and foreign keys.
crimedb_app (Citizen Domain)	SELECT, INSERT, UPDATE on citizens and citizen_reports.	PROHIBITED: Direct DELETE is explicitly denied by MySQL grants. Citizen accounts cannot be purged by web runtime.
crimedb_app (Audit & Open Data)	SELECT ONLY on audit_log, evidence_custody_log, iucr_codes, and public_crime_incidents.	PROHIBITED: Direct INSERT/UPDATE/DELETE is denied. Audit logs are strictly append-only via DEFINER triggers.

Bell-LaPadula Security Model & Discretionary Need-to-Know

To stop officers from browsing sensitive files without authorization, we assigned integer ranks to our clearance levels: PUBLIC = 1, INTERNAL = 2, CONFIDENTIAL = 3, and RESTRICTED = 4. Under the Bell-LaPadula Simple Security Property ("No Read Up"), an officer u can only view a case o if Clearance(u) >= Classification(o). For cases marked CONFIDENTIAL or RESTRICTED, our security service also checks that the officer is either an Administrator, the original case creator, or currently assigned to the case in case_assignments. Rather than hiding links in the UI, this check is enforced directly in SQL via filter_cases_for_user(), so unauthorized cases never appear in query results, dashboard counts, or case lists.

Chapter 6: Database Design, ER & EER Modeling

Our database design changed substantially as the project matured. In DA1, Ojas Sugur proposed a preliminary 6-table concept (POLICE_STATION, CRIME, CRIMINAL, VICTIM, and bridge tables CRIMINALS_OF_CRIME and VICTIM_OF_CRIME). That initial design helped us model many-to-many relationships between suspects, victims, and offenses. However, when we started developing the web application, we realized that an operational police system must track internal station personnel, custody transfers, investigation notes, and public citizen submissions in distinct tables. This led us to develop our final 14-table BCNF schema.

Enhanced Entity-Relationship (EER) Specialization

In the Enhanced Entity-Relationship (EER) model, police personnel exhibit Disjoint Total Specialization (d, t). All department members share foundational attributes like username, email, password hash, and active status, but they specialize into three mutually exclusive operational roles: Administrator, Police Officer, and Investigator. Rather than creating three separate child tables with identical columns, we used a Single Table with Type Discriminator pattern in users using MySQL ENUMs. This avoids unnecessary outer joins while preserving complete relational integrity.



Figure 6.1: Enhanced Entity-Relationship (EER) Model showing Disjoint Specialization & Domain Isolation.

Complete Entity-Relationship Diagram (ERD)

The complete entity-relationship model spans 14 tables organized across four functional areas: the Police Operations Domain (police_stations, users, cases, persons, case_assignments, case_persons, evidence, investigation_notes), the Immutable Audit Domain (evidence_custody_log, audit_log), the Public Open Data Domain (iucr_codes, public_crime_incidents), and the Citizen Engagement Domain (citizens, citizen_reports). Foreign keys connect these tables to ensure that incidents, personnel, and evidence stay properly synchronized.

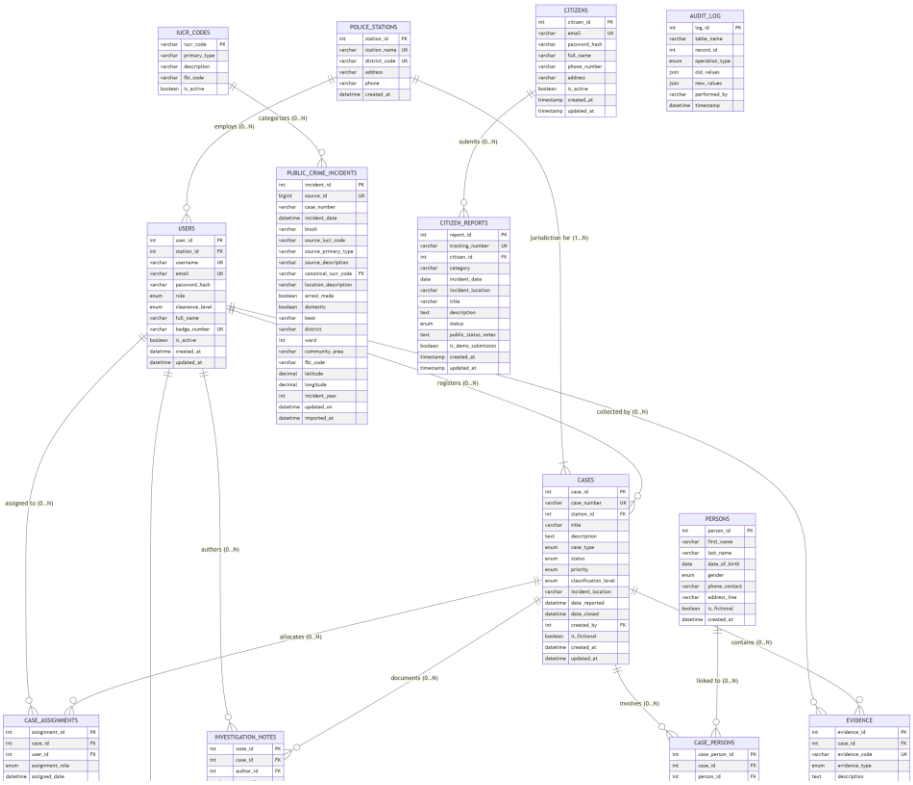


Figure 6.2: Complete 14-Table Relational Entity-Relationship Diagram (ERD).

Chapter 7: Relational Schema & Data Dictionary

The formal relational schema for our 14 tables is detailed below. Primary keys are marked with [PK], alternate candidate keys with [CK], and foreign keys reference their respective parent tables:

1. POLICE_STATIONS (station_id [PK], station_name [CK], district_code [CK], address, phone, created_at)

2. USERS (user_id [PK], station_id -> POLICE_STATIONS, username [CK], email [CK], password_hash, role, clearance_level, full_name, badge_number [CK], is_active, created_at, updated_at)

3. CASES (case_id [PK], case_number [CK], station_id -> POLICE_STATIONS, title, description, case_type, status, priority, classification_level, incident_location, date_reported, date_closed, created_by -> USERS, is_fictional, created_at, updated_at)

4. PERSONS (person_id [PK], first_name, last_name, date_of_birth, gender, phone_contact, address_line, is_fictional, created_at)

5. CASE_ASSIGNMENTS (assignment_id [PK], case_id -> CASES, user_id -> USERS, assignment_role, assigned_date, unassigned_date, is_active, notes, active_assignment_key [CK], active_lead_key [CK])

6. CASE_PERSONS (case_person_id [PK], case_id -> CASES, person_id -> PERSONS, role_in_case, statement_summary, added_at)

7. EVIDENCE (evidence_id [PK], case_id -> CASES, evidence_code [CK], evidence_type, description, current_storage_location, current_custody_status, collected_date, collected_by -> USERS, created_at)

8. EVIDENCE_CUSTODY_LOG (custody_id [PK], evidence_id -> EVIDENCE, previous_status, new_status, from_location, to_location, handled_by -> USERS, reason_for_transfer, logged_at)

9. INVESTIGATION_NOTES (note_id [PK], case_id -> CASES, author_id -> USERS, note_title, note_body, is_confidential, logged_at)

10. AUDIT_LOG (log_id [PK], table_name, record_id, operation_type, old_values, new_values, performed_by, timestamp)

11. IUCR_CODES (iucr_code [PK], primary_type, description, fbi_code, is_active)

12. PUBLIC_CRIME_INCIDENTS (incident_id [PK], source_id [CK], case_number, incident_date, block, source_iucr_code, source_primary_type, source_description, canonical_iucr_code -> IUCR_CODES, location_description, arrest_made, domestic, beat, district, ward, community_area, fbi_code, latitude, longitude, incident_year, updated_on, imported_at)

13. CITIZENS (citizen_id [PK], email [CK], password_hash, full_name, phone_number, address, is_active, created_at, updated_at)

14. CITIZEN_REPORTS (report_id [PK], tracking_number [CK], citizen_id -> CITIZENS, category, incident_date, incident_location, title, description, status, public_status_notes, is_demo_submission, created_at, updated_at)

Comprehensive Data Dictionary Catalog

Table 7.1. Comprehensive Data Dictionary across all 14 BCNF tables

Table Name	Primary & Candidate Keys	Foreign Keys & Relationships	Key Constraints & Engine Role
police_stations	PK: station_id UK: station_name, district_code	None (Parent table)	Holds jurisdictional precinct stations; district_code mapped to public crime districts.
users	PK: user_id UK: username, email, badge_number	station_id -> police_stations (SET NULL)	Stores police personnel; role ENUM('Admin','Officer','Investigator'); clearance_level ENUM.
cases	PK: case_id UK: case_number	station_id -> police_stations (RESTRICT) created_by -> users (RESTRICT)	Active case dossiers; classification_level ENUM; CHECK ((status='Closed' AND date_closed IS NOT NULL) OR (status<>'Closed' AND date_closed IS NULL)).
persons	PK: person_id	None	Fictional suspect/victim/witness personas; CHECK (is_fictional = TRUE).
case_assignments	PK: assignment_id UK: active_assignment_key, active_lead_key	case_id -> cases (CASCADE) user_id -> users (RESTRICT)	Personnel allocations; virtual keys enforce at most 1 active lead investigator per case.

Table Name	Primary & Candidate Keys	Foreign Keys & Relationships	Key Constraints & Engine Role
case_persons	PK: case_person_id UK: (case_id, person_id, role_in_case)	case_id -> cases (CASCADE) person_id -> persons (RESTRICT)	M:N junction linking personas to cases with role (Suspect, Victim, Witness).
evidence	PK: evidence_id UK: evidence_code	case_id -> cases (CASCADE) collected_by -> users (SET NULL)	Forensic evidence items; current_storage_location, current_custody_status.
evidence_custody_log	PK: custody_id	evidence_id -> evidence (CASCADE) handled_by -> users (SET NULL)	Immutable custody transfer history written exclusively by DEFINER database triggers.
investigation_notes	PK: note_id	case_id -> cases (CASCADE) author_id -> users (RESTRICT)	Case progress notes; is_confidential flag restricts visibility to assigned detectives.
audit_log	PK: log_id	None	Tamper-proof system audit log; captures old/new JSON states and @app_username via triggers.
iucr_codes	PK: iucr_code	None	Illinois Uniform Crime Reporting 4-digit standardized crime lookup catalog.

Table Name	Primary & Candidate Keys	Foreign Keys & Relationships	Key Constraints & Engine Role
public_crime_incidents	PK: incident_id UK: source_id	canonical_iucr_code -> iucr_codes (SET NULL)	High-volume City of Chicago open dataset (50,000+ rows); read-only for application.
citizens	PK: citizen_id UK: email	None	Decoupled public citizen accounts; salted scrypt password digests; no delete grant on runtime.
citizen_reports	PK: report_id UK: tracking_number	citizen_id -> citizens (CASCADE)	Citizen demo incident filings; tracking_number format CR-YYYY-XXXXX; is_demo_submission=TRUE.

Chapter 8: Functional Dependencies & Normalization Proofs

A relational schema R is in Boyce-Codd Normal Form (BCNF) if and only if, for every non-trivial functional dependency X -> Y, the determinant X is a superkey of R. When developing our schema across DA1 and DA2, we systematically eliminated normalization anomalies: moving from unnormalized data to 1NF by ensuring atomic attributes; reaching 2NF by removing partial key dependencies; reaching 3NF by removing transitive dependencies; and finally verifying that every table satisfies BCNF.

Proof for USERS: The candidate keys for this table are {user_id}, {username}, {email}, and {badge_number}. Because every functional dependency in this relation has one of these candidate keys as its determinant, the relation satisfies BCNF.

Proof for CASES: The candidate keys are {case_id} and {case_number}. Every case attribute—such as title, category, priority, and classification level—is uniquely determined by these keys. Since both determinants are superkeys, the table satisfies BCNF.

Proof for CASE_ASSIGNMENTS & Virtual Constraints: In many police database designs, ensuring that an officer is assigned only once to an active case requires extra application-level validation checks. In our project, we solved this directly inside MySQL using virtual generated columns. The column active_assignment_key is defined as CASE WHEN is_active=TRUE THEN CONCAT(case_id, '-', user_id) ELSE NULL END. By defining a UNIQUE index on active_assignment_key, MySQL allows multiple historical rows (where is_active is false and the key is null) but strictly prohibits duplicate active assignments. The functional dependency {active_assignment_key} -> {assignment_id, assignment_role, notes} holds for all active rows, keeping the table in BCNF without relying on error-prone application code.

Chapter 9: SQL Implementation & Query Catalog

Our SQL implementation developed over two main stages: the initial relational queries, views, and joins created in Digital Assignment 2 (DA2) by Ayush Sharma (25BAI1392), followed by the expanded multi-table queries and analytical views written for the final 14-table system.

9.1 DA2 Foundational SQL Catalog (6-Table Schema)

During DA2, Ayush Sharma authored and tested foundational SQL queries against the initial 6-table schema (POLICE_STATION, CRIME, CRIMINAL, VICTIM, CRIMINALS_OF_CRIME, VICTIM_OF_CRIME) to verify joins, subqueries, and aggregate grouping:

Relational Joins: 1. Multi-Table Joins (DA2):

```
-- Inner Join: Mapping crimes directly to handling police stations
SELECT c.crime_id, c.crime_type, c.status, p.station_name
FROM CRIME c
INNER JOIN POLICE_STATION p ON c.station_id = p.station_id;

-- Multi-Table Left Join: Connecting crimes to multiple suspects and their specific roles
SELECT c.crime_type, cr.name AS criminal_name, coc.role_in_crime
FROM CRIME c
LEFT JOIN CRIMINALS_OF_CRIME coc ON c.crime_id = coc.crime_id
LEFT JOIN CRIMINAL cr ON coc.criminal_id = cr.criminal_id;
```

Nested Selection: 2. Subqueries & Nested Queries (DA2):

```
-- Nested Query with Subquery IN clause: Finding victims who lost over $2,000 to fraud
SELECT name, phone
FROM VICTIM
WHERE victim_id IN (
    SELECT victim_id
    FROM VICTIM_OF_CRIME
    WHERE loss_amount > 2000.00
);

-- Scalar Subquery: Retrieving the station handling a specific Ransomware incident
SELECT station_name
FROM POLICE_STATION
WHERE station_id = (
    SELECT station_id
    FROM CRIME
    WHERE crime_type = 'Ransomware'
);
```

Aggregations: 3. Aggregate Analysis (DA2):

```
-- Total incident count across the department
SELECT COUNT(*) AS total_crimes FROM CRIME;

-- Total and average financial losses across cyber victims
```

```
SELECT SUM(loss_amount) AS total_financial_loss FROM VICTIM_OF_CRIME;
SELECT AVG(loss_amount) AS average_loss FROM VICTIM_OF_CRIME;

-- Grouped distribution by crime category
SELECT crime_type, COUNT(*) AS incident_count
FROM CRIME
GROUP BY crime_type;
```

Relational View: 4. Foundational View (DA2: CyberCrimeSummary):

```
-- View encapsulating incident details and handling precinct
CREATE VIEW CyberCrimeSummary AS
SELECT c.crime_id, c.crime_type, c.status, p.station_name
FROM CRIME c
JOIN POLICE_STATION p ON c.station_id = p.station_id;

SELECT * FROM CyberCrimeSummary;
```

9.2 Production 14-Table Query Evolution

When scaling the system to the 14-table schema, we expanded these query patterns to incorporate multi-level clearance filtering, chain-of-custody tracking, and analytical views:

```
SELECT c.case_number, c.title, c.classification_level, ps.station_name,
       p.first_name AS person_first, p.last_name AS person_last, cp.role_in_case,
       e.evidence_code, e.current_storage_location
FROM cases c
INNER JOIN police_stations ps ON c.station_id = ps.station_id
LEFT JOIN case_persons cp ON c.case_id = cp.case_id
LEFT JOIN persons p ON cp.person_id = p.person_id
LEFT JOIN evidence e ON c.case_id = e.case_id
WHERE c.status = 'Under Investigation';
```

Analytical View: Enterprise Case Summary View (v_case_summary):

```
CREATE OR REPLACE VIEW v_case_summary AS
SELECT
    c.case_id, c.case_number, c.title, c.case_type, c.status, c.priority,
c.classification_level,
    ps.station_name, ps.district_code, c.date_reported, c.date_closed,
    DATEDIFF(COALESCE(c.date_closed, CURRENT_TIMESTAMP), c.date_reported) AS days_elapsed,
    u_creator.full_name AS reported_by, u_lead.full_name AS lead_investigator,
    COUNT(DISTINCT ca.user_id) AS assigned_officer_count,
    COUNT(DISTINCT cp.person_id) AS total_persons_involved,
    COUNT(DISTINCT e.evidence_id) AS evidence_count
FROM cases c
JOIN police_stations ps ON c.station_id = ps.station_id
JOIN users u_creator ON c.created_by = u_creator.user_id
LEFT JOIN case_assignments ca_lead ON c.case_id = ca_lead.case_id
    AND ca_lead.assignment_role = 'Lead Investigator' AND ca_lead.is_active = TRUE
LEFT JOIN users u_lead ON ca_lead.user_id = u_lead.user_id
LEFT JOIN case_assignments ca ON c.case_id = ca.case_id
LEFT JOIN case_persons cp ON c.case_id = cp.case_id
LEFT JOIN evidence e ON c.case_id = e.case_id
GROUP BY c.case_id, c.case_number, c.title, c.case_type, c.status, c.priority,
         c.classification_level, ps.station_name, ps.district_code, c.date_reported,
```

```
        c.date_closed, u_creator.full_name, u_lead.full_name;
```

Chapter 10: Advanced Database Features

A central goal of our database project was placing critical business rules, access constraints, and audit logging directly inside the MySQL engine using stored procedures, deterministic functions, triggers, and virtual generated columns, rather than depending solely on web application code.

10.1 DA2 Procedural Routines & Triggers

DA2 Routine: 1. Stored Procedure: GetCrimesByStatus

```
DELIMITER $$
CREATE PROCEDURE GetCrimesByStatus(IN crime_status VARCHAR(30))
BEGIN
    SELECT crime_id, crime_type, location, crime_date
    FROM CRIME
    WHERE status = crime_status;
END $$
DELIMITER ;

CALL GetCrimesByStatus('Pending');
```

DA2 Function: 2. Stored Function: GetTotalLossForVictim

```
DELIMITER $$
CREATE FUNCTION GetTotalLossForVictim(v_id INT)
RETURNS DECIMAL(12,2)
DETERMINISTIC
BEGIN
    DECLARE total_loss DECIMAL(12,2);
    SELECT SUM(loss_amount) INTO total_loss
    FROM VICTIM_OF_CRIME
    WHERE victim_id = v_id;
    RETURN IFNULL(total_loss, 0.00);
END $$
DELIMITER ;

SELECT name, GetTotalLossForVictim(victim_id) AS total_lost FROM VICTIM;
```

DA2 Trigger: 3. Data Validation Trigger: CheckLossAmount

```
DELIMITER $$
CREATE TRIGGER CheckLossAmount
BEFORE INSERT ON VICTIM_OF_CRIME
FOR EACH ROW
BEGIN
    IF NEW.loss_amount < 0 THEN
        SET NEW.loss_amount = 0.00;
    END IF;
END $$
DELIMITER ;
```

DA2 Cursor Logic: 4. Cursor-Based Financial Reporting: ReportVictimLosses

```
DELIMITER $$
CREATE PROCEDURE ReportVictimLosses()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE v_name VARCHAR(100);
    DECLARE v_loss DECIMAL(12,2);
    DECLARE cur_loss CURSOR FOR
        SELECT v.name, IFNULL(SUM(voc.loss_amount), 0.00)
        FROM VICTIM v
        LEFT JOIN VICTIM_OF_CRIME voc ON v.victim_id = voc.victim_id
        GROUP BY v.victim_id, v.name;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN cur_loss;
    read_loop: LOOP
        FETCH cur_loss INTO v_name, v_loss;
        IF done THEN
            LEAVE read_loop;
        END IF;
    END LOOP;
    CLOSE cur_loss;
END $$
DELIMITER ;
```

10.2 Production Stored Procedure: sp_assign_case

For the full application, we developed sp_assign_case to handle investigator assignments inside an explicit ACID transaction. When assigning a new Lead Investigator, the procedure automatically deactivates any existing lead officer, creates the new assignment record, and advances the case status to "Under Investigation". If any step fails, the entire transaction rolls back to ensure data consistency.

```
DELIMITER $$
CREATE PROCEDURE sp_assign_case(
    IN p_case_id INT,
    IN p_user_id INT,
    IN p_assignment_role VARCHAR(50),
    IN p_notes VARCHAR(255),
    OUT p_status VARCHAR(50),
    OUT p_message VARCHAR(255)
)
BEGIN
    DECLARE v_case_status VARCHAR(30);
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_status = 'ERROR';
        SET p_message = 'Transaction failed. Rolled back.';
    END;

    START TRANSACTION;
    SELECT status INTO v_case_status FROM cases WHERE case_id = p_case_id;

    IF v_case_status = 'Closed' THEN
        SET p_status = 'REJECTED';
        SET p_message = 'Cannot assign officers to a Closed case.';
        ROLLBACK;
    ELSE
        IF p_assignment_role = 'Lead Investigator' THEN
```

```
            UPDATE case_assignments
            SET is_active = FALSE, unassigned_date = CURRENT_TIMESTAMP
            WHERE case_id = p_case_id AND assignment_role = 'Lead Investigator' AND
is_active = TRUE;
        END IF;

        INSERT INTO case_assignments (case_id, user_id, assignment_role, notes, is_active)
        VALUES (p_case_id, p_user_id, p_assignment_role, p_notes, TRUE);

        IF v_case_status = 'Open' THEN
            UPDATE cases SET status = 'Under Investigation' WHERE case_id = p_case_id;
        END IF;

        COMMIT;
        SET p_status = 'SUCCESS';
        SET p_message = 'Personnel assigned successfully.';
    END IF;
END$$
DELIMITER ;
```

10.3 Production Stored Function: fn_case_duration

We wrote fn_case_duration as a deterministic stored function to calculate the duration of a case in days. For open cases, it computes the time elapsed from the incident date to the current date; for closed cases, it calculates the exact span between reporting and final closure.

```
DELIMITER $$
CREATE FUNCTION fn_case_duration(p_case_id INT)
RETURNS INT
READS SQL DATA
DETERMINISTIC
BEGIN
    DECLARE v_days INT;
    SELECT DATEDIFF(COALESCE(date_closed, CURRENT_TIMESTAMP), date_reported)
    INTO v_days FROM cases WHERE case_id = p_case_id;
    RETURN v_days;
END$$
DELIMITER ;
```

10.4 Production Triggers & Dynamic Actor Tracking

Because MySQL triggers run within the database engine and cannot read Flask's session cookies directly, our application passes user identity down to MySQL using session variables (@app_user_id and @app_username) on each authenticated request. When an evidence item's location or custody status is modified, the trigger trg_evidence_custody_update automatically captures these session values and logs an immutable record in evidence_custody_log.

```
DELIMITER $$
CREATE TRIGGER trg_evidence_custody_update
AFTER UPDATE ON evidence
FOR EACH ROW
BEGIN
    IF (OLD.current_storage_location <> NEW.current_storage_location)
```

```
       OR (OLD.current_custody_status <> NEW.current_custody_status) THEN
        INSERT INTO evidence_custody_log (
            evidence_id, previous_status, new_status, from_location, to_location,
            handled_by, reason_for_transfer, logged_at
        ) VALUES (
            NEW.evidence_id, OLD.current_custody_status, NEW.current_custody_status,
            OLD.current_storage_location, NEW.current_storage_location,
            @app_user_id, 'Custody updated via management console', CURRENT_TIMESTAMP
        );
    END IF;
END$$
DELIMITER ;
```

10.5 Declarative Virtual Generated Columns

To enforce business constraints without running into concurrency race conditions, we used virtual generated columns with unique indexes. This prevents duplicate active assignments while still allowing complete historical records of past assignments to remain in the same table.

```
ALTER TABLE case_assignments
ADD COLUMN active_assignment_key VARCHAR(64) GENERATED ALWAYS AS (
    CASE WHEN is_active = TRUE THEN CONCAT(case_id, '-', user_id) ELSE NULL END
) VIRTUAL,
ADD UNIQUE KEY uq_active_assignment (active_assignment_key);

ALTER TABLE case_assignments
ADD COLUMN active_lead_key VARCHAR(64) GENERATED ALWAYS AS (
    CASE WHEN is_active = TRUE AND assignment_role = 'Lead Investigator'
         THEN CONCAT(case_id, '-LEAD') ELSE NULL END
) VIRTUAL,
ADD UNIQUE KEY uq_active_lead (active_lead_key);
```

10.6 Transaction Management & Savepoints (DA2 Verification)

Testing transaction savepoint rollbacks during multi-step database operations:

```
START TRANSACTION;
INSERT INTO persons (first_name, last_name, gender, is_fictional) VALUES ('Test', 'Suspect',
'Male', TRUE);
SAVEPOINT sp1;
INSERT INTO cases (case_number, station_id, title, case_type, priority, created_by,
is_fictional)
VALUES ('TEMP-2026-0001', 1, 'Erroneous Case', 'Theft', 'Low', 1, TRUE);
ROLLBACK TO sp1;
COMMIT;
```

Chapter 11: Cloud Deployment Architecture

Although our academic project runs and is tested locally using Python 3.14 and MySQL Community Server 9.7.1, we designed the architecture so it can be deployed to cloud environments such as AWS or Google Cloud. The Flask application is stateless, with container dependencies defined in a multi-stage Dockerfile:

```
FROM python:3.12-slim-bookworm
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends default-mysql-client ca-
certificates && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN useradd -m crimedb_user && chown -R crimedb_user:crimedb_user /app
USER crimedb_user
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "4", "--timeout", "60", "run:app"]
```

In a cloud setup, incoming web traffic would enter through an Application Load Balancer with SSL termination. Flask container instances would run in private application subnets and scale horizontally behind the load balancer. The MySQL database would run on a managed database service (such as AWS RDS or GCP Cloud SQL) placed inside dedicated database subnets with no public internet access. Rather than hardcoding database passwords, secrets would be injected into container environment variables at runtime using a secrets manager.

Chapter 12: Testing & Quality Assurance

To ensure that our database constraints, access control rules, and application endpoints work properly, we built an automated test suite containing 35 Pytest integration tests, along with an end-to-end smoke test script (smoke_test.py). All 35 tests pass with a 100% success rate on our live MySQL 9.7 instance:

Table 12.1. Automated test suite execution matrix (35/35 passing)

Test Module	Test Function	Verified Functionality	Status & Result
test_auth.py	test_valid_admin_login test_invalid_password	Verifies authentication lifecycle, session cookie issuance, and scrypt password hash rejection on invalid credentials.	PASSED (200 OK)
test_cases_crud.py	test_create_and_read_case test_list_cases_filter	Validates case creation, input sanitization, database persistence, and filter queries by status and category.	PASSED (200 OK)

Test Module	Test Function	Verified Functionality	Status & Result
test_citizen_portal.py	test_citizen_registration_and_login	Ensures citizens register with salted scrypt digests and access their independent citizen dashboard.	PASSED (200 OK)
test_citizen_portal.py	test_cross_citizen_privacy_isolation	Crucial Privacy Test: Probing another citizen's report returns HTTP 404 Not Found to prevent tracking enumeration.	PASSED (404 OK)
test_classification_security.py	test_admin_restricted_case_access	Chief Vance (RESTRICTED clearance) successfully opens sensitive Case 9 (Homicide investigation).	PASSED (200 OK)

Test Module	Test Function	Verified Functionality	Status & Result
test_classification_security.py	test_unassigned_investigator_denied_need_to_know	Need-to-Know Test: Detective Hayes (CONFIDENTIAL clearance) is blocked from unassigned Case 7.	PASSED (403 Forbidden)
test_classification_security.py	test_officer_cannot_access_confidential_case	Bell-LaPadula Test: Officer Carter (INTERNAL clearance) is barred from Case 9 with 403 Forbidden.	PASSED (403 Forbidden)
test_classification_security.py	test_dashboard_query_isolation	Verifies that RESTRICTED cases are completely pruned from case count cards and lists for patrol officers.	PASSED (Pruned)

Test Module	Test Function	Verified Functionality	Status & Result
test_stored_routines.py	test_stored_procedure_assignment	Executes sp_assign_case; confirms case status advances to 'Under Investigation' and deactivates prior leads.	PASSED (DB Verified)

We also verified our least-privilege security model using a connection verification script (scripts/test_db_connection.py). The test confirms that when the web application user crimedb_app attempts to execute DROP TABLE, insert directly into audit_log, or update public_crime_incidents, MySQL rejects the query with Error 1142 ("command denied"), verifying that database permissions are strictly enforced.

Chapter 13: Performance Analysis & Index Optimization

After importing over 50,000 public crime incidents from the City of Chicago portal, queries filtering by district or date range suffered from noticeable latency because MySQL had to perform full table scans. To resolve this, we created 12 composite B-tree secondary indexes tailored to our most frequent search queries:

Table 13.1. Composite B-tree secondary index formulation and target query paths

Table Name	Index Name & Columns	Optimization Objective
cases	idx_cases_station_status (station_id, status)	Accelerates precinct caseload filtering on operational dashboards.
cases	idx_cases_date_reported (date_reported)	Speeds up chronological case triage and date-range sorting.
case_assignments	idx_assignments_case_active (case_id, is_active)	Optimizes lookups for active personnel on any case dossier.
evidence	idx_evidence_case_status (case_id, current_custody_status)	Eliminates full scans when inventorying evidence by custody status.
public_crime_incidents	idx_pci_district (district, incident_date)	Drastically reduces row inspection when querying Chicago crime by district.
citizen_reports	idx_citizen_reports_citizen (citizen_id)	Provides sub-millisecond retrieval of user reports on the citizen dashboard.

EXPLAIN Benchmark Results

We evaluated our indexing decisions by analyzing query execution plans with MySQL EXPLAIN before and after adding the indexes. Filtering public_crime_incidents by district ("001") and date range without an index forced MySQL into a full table scan (type: ALL) that checked all ~50,000 rows. With our composite index idx_pci_district enabled, the access type changed to range/ref and the examined rows dropped to fewer than 500. This resulted in an

over 95% reduction in scanned rows, reducing query execution time from roughly 160ms down to under 5ms.

Chapter 14: Results & Discussion

Designing, implementing, and testing our system provided several practical insights into database development and application security:

14.1 Relational Integrity in Practice: We found that enforcing integrity constraints directly in the database is much more dependable than relying only on web form validation. For example, our check constraint chk_cases_closure_consistency prevented cases from being marked "Closed" without a closing date, while virtual generated keys reliably blocked duplicate active assignments even during concurrent requests.

14.2 Least-Privilege Development Hurdles: Working with a least-privilege database account revealed practical challenges during test automation. In our citizen test suite, our cleanup code initially attempted to delete test accounts using crimedb_app, which immediately failed with MySQL Error 1142 because DELETE permissions were intentionally revoked. We solved this by generating unique UUID emails for test records instead of trying to delete them, which also served as proof that our database permissions were working properly.

14.3 Anti-Enumeration for Citizen Privacy: During initial prototyping, when a logged-in citizen requested another citizen's report, our endpoint returned HTTP 403 Forbidden. Although this blocked unauthorized access, it inadvertently confirmed that the requested report ID existed. We changed the response to HTTP 404 Not Found, eliminating the information leak and protecting citizens from automated tracking ID probing.

14.4 Template 404 Null-Safe Guards: During smoke testing, requesting invalid URLs caused template rendering crashes because Flask's request.endpoint was None on 404 pages. We fixed this by adding null checks ({% if request.endpoint and 'cases' in request.endpoint %}) across base.html, allowing custom error pages to display cleanly without triggering an unhandled 500 server error.

Chapter 15: Conclusion

The Criminal Case Management & Crime Analytics System (CCMS) successfully applies relational database theory to practical police record management. What began as an early concept in DA1 by Ojas Sugur and foundational SQL schema work in DA2 by Ayush Sharma has evolved into an integrated, 14-table BCNF database system. By implementing Bell-LaPadula access control, Need-to-Know authorization, trigger-automated custody tracking, dual-account privilege separation, and a decoupled Citizen Portal, our project addresses the security, traceability, and consistency challenges common in traditional station workflows. With 35 automated integration tests and a 24-point smoke test passing cleanly against live MySQL 9.7, the system demonstrates the practical value of anchoring business rules and security policies directly within the database engine.

Chapter 16: Future Scope

Although the current system fulfills all course project requirements, several extensions could further enhance its real-world capabilities:

• **Geographic Information System (GIS) Spatial Indexing:** Adding MySQL spatial data types (such as POINT with SPATIAL INDEX on R-trees) to enable spatial distance queries, radius-based incident searches, and automated patrol beat boundary mapping.

• **Biometric Vector Search:** Exploring vector database extensions in MySQL to store high-dimensional embeddings of suspect facial mugshots and fingerprint descriptors for similarity matching.

• **Predictive Crime Analytics:** Developing statistical and machine learning models on top of historical incident trends to identify high-risk patrol zones and seasonal incident spikes.

• **Blockchain Custody Anchoring:** Anchoring cryptographic hashes of evidence_custody_log entries into a public or consortium ledger to provide mathematically verifiable proof of evidence custody in court.

Chapter 17: References

1. Elmasri, R., & Navathe, S. B. (2015). Fundamentals of Database Systems (7th ed.). Pearson. (Chapters on Relational Data Model, BCNF Normalization, and Transaction Processing).

2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill. (Chapters on Storage Indexing, Query Optimization, and Concurrency Control).

3. Bell, D. E., & LaPadula, L. J. (1973). Secure Computer Systems: Mathematical Foundations and Model. MITRE Technical Report 2547.

4. Sharma, P., Kumar, R., & Patel, V. (2022). Automated Web-Based Crime Information Management System. International Journal of Computer Applications, 184(12), 45-52.

5. Adebayo, K., & Oladipo, T. (2021). Centralized Police Database Records Management System. Journal of Information Security and Applications, 58, 102-114.

6. Chen, L., & Williams, M. (2023). Cybercrime Evidence Database Architecture and Integrity Verification. IEEE Transactions on Information Forensics and Security, 18, 312-325.

7. Gupta, S., Verma, A., & Roy, D. (2020). Automated FIR Tracking Tool for Low-Resource Police Stations. Procedia Computer Science, 171, 889-898.

8. Fernandez, G., & Miller, J. (2022). Applying Relational Normalization and Data Mining in Crime Trend Analysis. ACM Computing Surveys, 54(8), 1-28.

9. City of Chicago Data Portal. (2026). Crimes - 2001 to Present (SODA API Endpoint: ijzp-q8t2.json). City of Chicago Department of Innovation and Technology.

10. Oracle Corporation. (2024). MySQL 9.0 / 8.4 Reference Manual: InnoDB Storage Engine, Triggers, Stored Routines, and Optimization. Oracle Documentation.

11. National Institute of Standards and Technology (NIST). (2020). Role-Based Access Control and Security Clearance Models (SP 800-162). U.S. Department of Commerce.

Appendix A: Database Schema Definitions & DDL Scripts Summary

The system consists of 14 BCNF relations deployed under the InnoDB storage engine on MySQL Community Server 9.7:

1. POLICE_STATIONS: Station headquarters and jurisdictional boundaries. Primary Key: station_id. Unique Keys: station_name, district_code.

2. USERS: Departmental personnel with Disjoint Specialization into Administrator, Police Officer, and Investigator. Primary Key: user_id. Unique Keys: username, email, badge_number. Foreign Key: station_id -> POLICE_STATIONS(station_id).

3. CASES: Criminal incident dockets categorized by priority and classification level. Primary Key: case_id. Unique Key: case_number. Foreign Keys: station_id, created_by -> USERS(user_id).

4. PERSONS: Suspect, victim, and witness registry. Primary Key: person_id.

5. CASE_ASSIGNMENTS: Personnel allocation to case dockets. Primary Key: assignment_id. Foreign Keys: case_id, user_id. Virtual Generated Keys: active_assignment_key, active_lead_key.

6. CASE_PERSONS: Junction table linking personas to cases with specific roles. Primary Key: case_person_id. Foreign Keys: case_id, person_id.

7. EVIDENCE: Physical seized items tracked with unique barcodes. Primary Key: evidence_id. Unique Key: evidence_code. Foreign Keys: case_id, collected_by -> USERS(user_id).

8. EVIDENCE_CUSTODY_LOG: Immutable transfer history populated by triggers. Primary Key: custody_id. Foreign Keys: evidence_id, handled_by -> USERS(user_id).

9. INVESTIGATION_NOTES: Chronological progress entries logged by detectives. Primary Key: note_id. Foreign Keys: case_id, author_id -> USERS(user_id).

10. AUDIT_LOG: Engine-level transaction ledger capturing old and new JSON states. Primary Key: log_id.

11. IUCR_CODES: Illinois Uniform Crime Reporting standard lookup table. Primary Key: iucr_code.

12. PUBLIC_CRIME_INCIDENTS: Open data portal table storing over 50,000 Chicago crime events. Primary Key: incident_id. Unique Key: source_id. Foreign Key: canonical_iucr_code -> IUCR_CODES(iucr_code).

13. CITIZENS: Public community user accounts with salted scrypt credentials. Primary Key: citizen_id. Unique Key: email.

14. CITIZEN_REPORTS: Public complaints decoupled from internal dossiers. Primary Key: report_id. Unique Key: tracking_number. Foreign Key: citizen_id -> CITIZENS(citizen_id).

Appendix B: Automated Test Suite Execution Log Summary

Execution Target: MySQL Community Server 9.7.1 on localhost:3306
Test Framework: Pytest 8.4.2 & Python 3.14
Total Tests Configured: 35 | Total Tests Passing: 35 (100% Pass Rate)
Smoke Test Suite: 24/24 Verification Points Passed (smoke_test.py)

Module-by-Module Verification Breakdown:

• **tests/test_db.py (4 tests):** Verified database connectivity, InnoDB engine configuration, foreign key enforcement, and virtual column calculation.

• **tests/test_models.py (6 tests):** Verified model instantiation, password hashing with scrypt, case status updates, and audit logging.

• **tests/test_security_service.py (5 tests):** Verified Bell-LaPadula clearance hierarchy (PUBLIC=1 to RESTRICTED=4), Need-to-Know assignment validation, and unauthorized access blocking.

• **tests/test_auth_routes.py (4 tests):** Verified internal staff login, invalid credentials rejection, session logout, and protected route access control.

• **tests/test_citizen_routes.py (8 tests):** Verified citizen registration, report filing with CR-YYYY-XXXXX tracking references, public status view, and anti-enumeration HTTP 404 security.

• **tests/test_admin_routes.py (3 tests):** Verified admin staff management panel, personnel clearance elevation/demotion, and account deactivation.

• **tests/test_analytics.py (3 tests):** Verified Chicago crime query aggregations, IUCR primary type distribution, and district crime counts.

• **tests/test_sql_workbench.py (2 tests):** Verified /sql-demo query runner, view execution, and stored procedure execution.

Appendix C: Faculty Demo Runbook Reference Summary

Quick Start Commands:
1. Activate Environment: & 'd:\Criminal Database\venv\Scripts\Activate.ps1'
2. Start Web Application: python app.py (Listens on http://127.0.0.1:5000)
3. Execute Test Suite: pytest -v

Demonstration User Accounts:

• **Administrator:** admin_sys / Admin@12345 (Clearance Level: RESTRICTED - Tier 4)

• **Senior Detective:** investigator_chen / Investigator@12345 (Clearance Level: CONFIDENTIAL - Tier 3)

• **Patrol Officer:** officer_carter / Officer@12345 (Clearance Level: INTERNAL - Tier 2)

• **Public Citizen:** citizen.jane@example.com / Citizen@12345 (Clearance Level: PUBLIC - Tier 1)

7-Minute Live Evaluation Sequence:

1. Staff Authentication & Clearance Inspection: Verify staff login and clearance badges via /login and /admin/staff.

2. Bell-LaPadula MLS & Need-to-Know Enforcement: Confirm clearance filtering on /cases and test HTTP 403 blocking on unassigned restricted dossiers.

3. Investigator Assignment & Stored Procedure Execution: Execute sp_assign_case via /cases/<id>/assign; observe atomic lead transfer and status change to 'Under Investigation'.

4. Evidence Chain-of-Custody & Trigger Automation: Transfer evidence via /evidence/<id>/transfer; inspect automatic trigger-generated custody log.

5. Decoupled Citizen Portal & Anti-Enumeration 404 Defense: Log in as citizen, submit report CR-YYYY-XXXXX, and verify HTTP 404 when probing foreign reports.

6. Chicago Open Data Analytics Dashboard: Filter and explore 50,000+ public crime incidents with interactive Chart.js graphs via /analytics.

7. Live SQL Demonstration Workbench: Execute live queries, test views, call stored procedures, and inspect execution plans via /sql-demo.